How we faked it

A dependent function $(p : P) \to R\,p$, in a language that has none

September 1, 2026

Abstract

Ghani's containers turn on a dependent function: the type of the reply depends on the prompt. TypeScript has no dependent types. This note is about the one substitution that makes the whole edifice stand — a generic method whose return type is a conditional type applied to the inferred type argument — what it genuinely buys, exactly where it degrades, and the one place the project abandons it altogether. Along the way it disposes of a natural suspicion: that `async` had something to do with it. It did not.

1 The thing to be faked

An inhabitant of a container (P, R) is a dependent function

$$(p : P) \to R\,p$$

in which the *type* of the result is computed from the *value* of the argument. Ask for a harvest and you get a harvest report; ask for a census and you get a census return; and no type in the language relates the two except through p itself.

TypeScript has no such construct. What it has is: generic functions, which infer a type argument from a value argument; and conditional types, which compute a type from a type. Put together they give this:

```
run<S extends Shape<C>>(shape: S): PosAt<C, S>
```

Three ingredients, and it is worth being precise about which does what.

generic inference	`S` is recovered from the argument — this is the only channel by which anything about p reaches the type level
conditional type	`PosAt<C, S>` computes the position type from `S`, by distributing over the union of fibres
a finite index	makes that distribution *exact*: over a finite S, $\Sigma\,(s : S).\,B\,s = \bigcup_s \{s\} \times B\,s$

2 `async` has nothing to do with it

The real signature carries a `Promise`, which invites the thought that concurrency is somehow load-bearing. It is not. Here is the same encoding with no `Promise`, no `await`, and no socket:

```
interface SyncRunner<C extends Cont> {
  readonly fibres?: C;
  run<S extends Shape<C>>(shape: S): PosAt<C, S>;    // no Promise
}
declare const agri: SyncRunner<Agri>;

const h = agri.run({tag: "Harvest", year: 1930});
const c = agri.run({tag: "Census"});
const grain: number = h.grain;        // typed HarvestReport, uncast
const hands: number = c.workforce;    // typed CensusReturn,  uncast
// @ts-expect-error   a Census reply has no ‘grain‘
const wrong: number = c.grain;
```

That file typechecks with zero errors, and the negative assertion fires — TypeScript reports an unused `@ts-expect-error` as an error in its own right, so its silence is evidence that the dependency really bites.

So why is the real thing `async`? Because the reply arrives from another process over a socket. Causation runs the other way from the suspicion: we chose real process boundaries, and *that* forced `Promise`. Ghani's own point in §3 is that what makes a program event-driven is the prompt-indexed response type, not the event loop — strip the waiting away and the container is what remains.

One thing `async` does buy. It makes ⊠ against ◁ visible in the *execution* and not only in the types. The tensor fires both prompts at once and requires both replies; the sequential composition cannot even construct its second prompt until the first reply is in hand:

```
const [l, r]  = await Promise.all([a.run(s.left, d), b.run(s.right, d)]);  // (x)
const first   = await a.run(s.first, d);
const second  = await b.run(s.next(first), d);                             //  <|
```

Synchronously these would be two calls in a row either way, and the difference would live entirely in the types. That is why the source says the `Promise.all` "is not an optimisation here; it is the definition".

3 Where the dependency is exact

The substitution works when the compiler can see *which fibre* a prompt lies in. Two cases, both verified:

```
const a = agri.run({tag: "Harvest", year: 1930});      // S = the literal shape
                                                       // -> {grain: number}   EXACT

const p: Shape<Agri> = {tag: "Harvest", year: 1930};   // declared as the union...
const b = agri.run(p);                                 // ...but narrowed by the
                                                       // discriminant -> EXACT
```

The second was a surprise. A `const` declared at the union type but initialised with a literal is narrowed by its discriminant tag, so the dependency survives being written down as a variable. The encoding is more robust than it looks.

The introduction rules do the real work

Each combinator ships a constructor for its shapes, and each is generic in the fibre. That is what keeps `S` narrow through a composite. The sequential one is the deepest instance:

```
step<SA extends Shape<A>, SB extends Shape<B>>(
  first: SA,
  next: (r: PosAt<A, SA>) => SB,
): {first: SA; next: (r: PosAt<A, SA>) => SB}
```

`SA` is inferred from `first`, so `next` is contextually typed at *that fibre alone*: it receives the positions of the shape you actually gave, not a union over all of them. This is the Σ-type faked — not a pair of prompts, but a prompt together with a function from its replies to the next prompt. And it is why, in the game, `supplied.first.toPort` is a `HaulReport` field with no cast at the call site.

4 Where it degrades

The fake carries the dependency on the **static type** of the argument, never on its runtime value. So it fails precisely where the compiler cannot see a single fibre — and the commonest such place is a *forwarder*: any helper generic over the container's prompts.

```
function forward(shape: Shape<Agri>) {
  return agri.run(shape);          // S is now the WHOLE union
}
const c = forward({tag: "Harvest", year: 1930});
// c : {grain: number} | {workforce: number}      <-- the dependency is gone
```

`PosAt` distributes over `S` as well as over `C`, so a union argument yields the union of *all* positions. Note what this is and is not. It is not unsound — you cannot read `grain` off it without narrowing first, and the compiler will stop you. But the dependency has been traded for a disjunction, and the caller must re-discover by hand what the type system knew one frame earlier.

This is the precise sense in which it is a fake. A genuine $(p : P) \to R\,p$ returns $R\,p$ for the actual runtime p, always, including inside a forwarder. This returns the exact position only where the compiler statically knows the fibre, and the union everywhere else. The dependency is on the singleton type TypeScript infers, not on the value.

5 Where it is abandoned

At the outermost edge the project gives up on it entirely. Everything a human or a search program sends to the Plan goes through:

```
export async function ask(prompt: unknown): Promise<Dict>
```

`unknown` in, `Record<string, unknown>` out. Not a degraded dependency — no typing at all.

This is not laziness, and it is worth saying why. An interactive menu is a forwarder by construction: it holds a list of options the player might choose, it does not know which until a key is pressed, and a list of ten differently typed prompts is exactly the union case above. The same is true of the letter search, which replays arbitrary sequences. So the container discipline holds *inside* the tower, where Gosplan composes and calls its commissariats through `leaf` and the combinators, and stops at the boundary where prompts become data chosen at runtime.

Which is the honest shape of the result. Dependent typing survives exactly as far as the compiler can follow the control flow, and no further. The tower's interior is typed; its front door is not.

6 The rest of the fakery, briefly

The container as its graph. R cannot be a function from values to types, so a container is presented as the union of its fibres, one `Fib` per prompt. `Fib` is never constructed — it is a type-level record, the object language of $\mathrm{Fam}(\mathrm{Set}^{\mathrm{op}})$, taken apart only by `infer`.

The delegate leg as a CLI tool. $u : P \to Q$ is not a function call but a spawned process, `delegate.ts <port> <json> <depth>`, which knows nothing of any container: it carries text. Control genuinely leaves the process. The file is located by `new URL("./delegate.ts", import.meta.url)`, resolved at run time and never checked — which is how a clean typecheck once produced a playthrough that died on its first hop.

The wire. JSON has six kinds of value and no notion of types, so a tag is a claim rather than a fact, and a wrong one usually does not crash: a missing property reads as `undefined`, so the handler returns HTTP 200 with a well-formed reply meaning nothing. Both directions are therefore validated rather than cast — 56 field checks across the five servers, and per-fibre decoders on the way back.

7 The bill

site	casts	what is asserted
`leaf`	2	the shape has a `tag`; the decoded reply is the position
`sumC`	2	the shape is a tagged side; the reply is the position
`tensorC`	2	the shape is a pair; the pair of replies is the position
`productC`	2	the shape is a pair; `{side, value}` is the position
`seqC`	2	the shape is `{first, next}`; `{first, second}` is the position
total	**10**	all inside `lib/algebra.ts`, 260 lines

Every `run` body must destructure a shape it knows only through a conditional type and re-assert the result; hence two per combinator. **None is in the game.** The game's own casts are `as const` on literals and narrowing strings a validator has already checked; no call site in `stalin` ever asserts the type of a container position.

Two traps, both silent

The fibre test written backwards. `Extract<C, Fib<S, unknown>>` keeps the fibres assignable *to* the query, but a shape you hold is usually *narrower* than its fibre's declared shape. What is wanted is `S extends S1`. Written the other way, every position becomes `never` and nothing complains near the mistake.

Conditional types are not inference sites. If `C` occurs only inside `Shape<C>` and `PosAt<C, S>`, TypeScript cannot infer it: `seqC(a, b)` silently infers `A = Cont` and every downstream position becomes `unknown`. Hence the phantom `readonly fibres?: C`, a field that is never assigned and exists solely to give `C` an inferable position.

Summary

Exact	inline prompts, and `const`s narrowed by their discriminant: the position type follows from the prompt, uncast
Degraded	any forwarder generic over prompts: the union of all positions, sound but no longer dependent
Abandoned	the outermost client, `ask(unknown): Dict` — because a menu is a forwarder by construction
Cost	10 casts, one phantom field, and a finite index
Irrelevant	`async`, which is about the socket